

Just Guess: Improved (Quantum) Algorithm for the Underdetermined MQ Problem

Alexander May , Massimo Ostuzzi , and Henrik Ressler

Ruhr University Bochum, Bochum, Germany

Abstract. We propose a novel algorithm to solve underdetermined systems of multivariate quadratic (MQ) equations over finite fields. In modern MQ signature schemes such as MAYO, QR-UOV and SNOVA finding solutions to such systems is equivalent to signature forgery.

The current benchmark for estimating the forgery bit complexity is Hashimoto’s algorithm which transforms the original underdetermined MQ system P into a more tractable system $\tilde{P}$. A hybrid combination of solving $\tilde{P}$ via Gröbner basis and exhaustive search eventually solves P . We introduce a novel transformation that pushes the hybrid approach to its extreme. Specifically, we reduce the underdetermined MQ system to a sequence of quadratic equations in a single variable at the cost of a larger exhaustive search. As a consequence, signature forgery no longer relies on the hardness of MQ solving but becomes pure guessing via exhaustive search. This in turn implies that signature forgery is significantly more vulnerable against quantum attacks via Grover search.

We provide accurate estimates for the classical and quantum bit complexity of forging signatures for MAYO, QR-UOV and SNOVA using our novel algorithm. We reduce the quantum security of all security levels of MAYO, QR-UOV and SNOVA.

Keywords: MQ signature forgery · MAYO · QR-UOV · SNOVA.

1 Introduction

Let $P : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ be a multivariate quadratic (MQ) map, i.e., a system of m quadratic polynomials that are evaluated in n variables $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{F}_q^n$ over some finite field $\mathbb{F}_q$. Without loss of generality, we restrict to *homogeneous* MQ maps, meaning that every monomial in every polynomial has degree exactly two. Let $\mathbf{t} \in \mathbb{F}_q^m$, which we call *target*. On input $(P, \mathbf{t})$, the MQ problem is to find a preimage of the target $\mathbf{t}$ under the map P , namely $\mathbf{x} \in \mathbb{F}_q^n$ such that $P(\mathbf{x}) = \mathbf{t}$.

We distinguish the following three different cases: *underdetermined*, *overdetermined* and *square* MQ problems, when $n > m$, $n < m$, and $n = m$, respectively. The square case is known to be an NP-hard problem [GJ79, BG94]. The overdetermined and underdetermined cases are widely used in *multivariate* public key cryptography, for encryption and digital signatures, respectively. In this paper, we focus on underdetermined MQ maps, i.e. with more variables n than equations m , that are central for the design of MQ-based digital signature schemes.

The scheme that inspired most MQ-based signatures is *Unbalanced Oil and Vinegar* (UOV) by Kipnis, Patarin and Goubin [KPG99], based on [Pat97]. UOV uses the hash-and-sign paradigm: to sign a message m , one has to find a preimage $\mathbf{x}$ of the target $\mathbf{t} = H(m)$ under the map P , where H is a cryptographic hash function. The public key is the MQ map P , which allows for efficient verification of $P(\mathbf{x}) = \mathbf{t}$. The secret key can be seen as a (linear and invertible) map $S : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ such that the MQ map $P \circ S$ can be easily inverted. On a high level, to sign with the secret key, one has to fix $n - m$ coordinates such that the map $P \circ S$ becomes linear in the remaining m variables.

Massively Underdetermined: the polynomial regime. While the secret key allows for efficient signature generation for $\mathbf{t} = H(m)$, an attacker willing to forge a signature is faced with an underdetermined MQ problem. A naive strategy randomly fixes $n - m$ variables to obtain a square MQ problem, and then uses a Gröbner Basis algorithm [MS17].

Kipnis, Patarin and Goubin [KPG99] showed that massively underdetermined MQ problems with $n \geq m(m + 1)$ can be solved in polynomial time. The idea of [KPG99] is to mimic the honest signing procedure and find a suitable linear change of variables $S : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ such that the MQ problem $(\tilde{P}, \mathbf{t}) = (P \circ S, \mathbf{t})$ is easier to solve than the MQ problem $(P, \mathbf{t})$. Once a solution $\mathbf{x} \in \mathbb{F}_q^n$ to the MQ problem $(\tilde{P}, \mathbf{t})$ is found, one can deduce the solution $S\mathbf{x} \in \mathbb{F}_q^n$ to the initial MQ problem.

Let us represent every homogeneous quadratic polynomial $p_1(\mathbf{x}), \dots, p_m(\mathbf{x})$ from P as upper-triangular matrices $P^{(i)}$ such that $\mathbf{x}^t P^{(i)} \mathbf{x} = p_i(\mathbf{x})$. After the transformation of [KPG99], the polynomials of $\tilde{P}$ have the form visualized in Figure 1. The last $n - m$ variables can now be fixed such that each polynomial depends only on the first m variables with square terms $x_1^2, \dots, x_m^2$ and no mixed terms $x_i x_j$, $i \neq j$. The remaining MQ problem can thus be linearised and solved in polynomial time by defining new variables $y_i := x_i^2$. Taking square roots of y_i yields the desired x_i . The algorithm by Kipnis, Patarin and Goubin was later improved by Miura, Hashimoto and Takagi [MHT13], by relaxing the requirement to half the number of variables $n \geq m(m + 1)/2$.

Moderately Underdetermined: the signature regime. The efficient algorithms in [KPG99, MHT13] do not encompass cryptographically relevant cases, since most multivariate schemes rely on more *moderately underdetermined* MQ problems. Thomae and Wolf [TW12] were the first to propose an algorithm for moderately underdetermined MQ problems that interpolates between the NP-hard case (square MQ problem) and the polynomial time case of a massively underdetermined MQ problem. Thomae-Wolf’s algorithm was improved and refined two years later by Cheng, Hashimoto, Miura and Takagi [CHMT14]. The strategy of [CHMT14] is again to compute the change of coordinates S , together with a system transformation $T : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$. Similarly to [KPG99], once a solution $\mathbf{x} \in \mathbb{F}_q^n$ to the easier MQ problem $(\tilde{P}, \mathbf{t}) = (T \circ P \circ S, T\mathbf{t})$ is found, one can deduce the solution $S\mathbf{x} \in \mathbb{F}_q^n$ to the initial MQ problem.

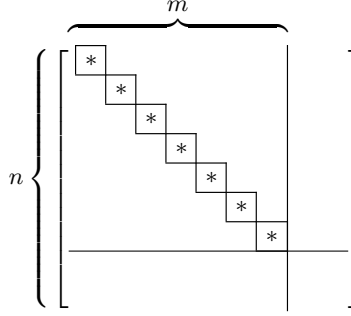


Fig. 1: Visualization of the polynomials after the transformation in [KPG99], for massively underdetermined MQ problems. Entries with $*$ are nonzero.

The transformation T should be thought of as a generalisation of Gaussian elimination that makes MQ problems easier by canceling the square terms $x_1^2, \dots, x_{m-b}^2$ for some parameter $0 \leq b \leq m$, see Figure 2 for a visualisation.

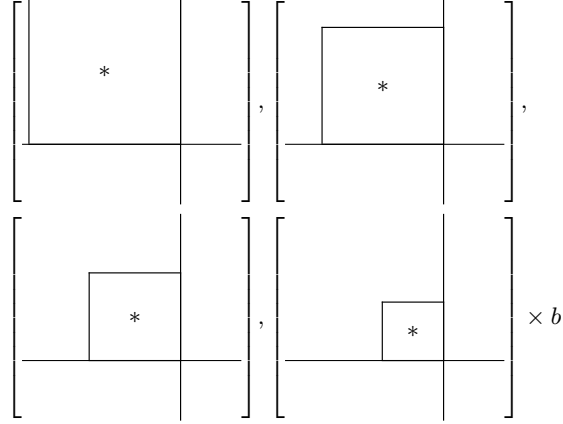


Fig. 2: Visualisation of the polynomials $p_1(\mathbf{x}), \dots, p_m(\mathbf{x})$ after the transformation in [CHMT14], for moderately underdetermined MQ problems.

While in the first $m-b$ matrices the size of the square block is decreasing one by one, the last b polynomials of $\tilde{P}$ all have the same shape, with a $b \times b$ square block of arbitrary entries. As a result, the time complexity of the [CHMT14] algorithm is dominated by solving an MQ problem with b equations in b variables over the field $\mathbb{F}_q$, which we denote by $\text{GB}(q, b, b)$. The desired form in Figure 2 can be achieved under the requirement $n \geq (m-b+1)(m+b)/2$, where one

chooses the smallest b satisfying this inequality. Notice that for the choice $b = 1$ we obtain the requirement $n \geq m(m + 1)/2$ of [MHT13]. While the algorithm by Cheng, Hashimoto, Miura and Takagi [CHMT14] nicely interpolates between the hard case and the polynomial time case, its complexity is concentrated in only one large block of equations.

Hashimoto's two-block strategy. Recently, Hashimoto [Has21] proposed a new algorithm for solving the MQ problem that balances and distributes the equations into two smaller (but still large) MQ subproblems. While Hashimoto's algorithm needs to compute only the variable change S , computing S itself requires to solve an MQ problem of complexity $\text{GB}(q, b, b)$. The polynomials of $\tilde{P}$ after the change of variables are visualised in Figure 3.

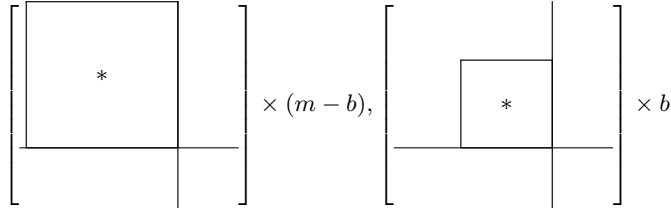


Fig. 3: Visualisation of the polynomials after the transformation in [Has21].

The shape of the first $m - b$ polynomials is not affected by the change of variables, while the last b polynomials have the same shape as in Figure 2. Overall, the complexity of Hashimoto's algorithm is $\text{GB}(q, b, b) + \text{GB}(q, m - b, m - b)$. While Hashimoto's algorithm nicely distributes the equations among two smaller MQ problems balancing their complexity, it does not interpolate between the polynomial time and the hard case.

Furue, Nakamura and Takagi [FNT21] showed that an exhaustive search of some variables can be combined with the aforementioned transformation strategies, resulting in a *hybrid* of MQ solving and exhaustive search. [Has21] exploits this hybrid technique, which leads to two additional benefits: the first MQ subproblem becomes overdetermined (which considerably reduces the complexity to find one of its solutions) and one of its quadratic equations can be completely linearised, which is beneficial for a Gröbner Basis algorithm. In this way, Hashimoto's algorithm relies on an optimisation problem: denoting by k the number of coordinates in an exhaustive search, the optimal choice of b and k minimises the cost

$$q^k \cdot (\text{GB}(q, m - k - b - 1, m - b - 1) + \text{GB}(q, b, b)) .$$

Our Results. In this paper, we propose a new algorithm that enhances the algorithm by [CHMT14] with the guessing strategy introduced by [FNT21] and an improved version of the linearisation by [Has21].

Our polynomial time transformation turns the underdetermined MQ problem into p (yellow) quadratic equations in one single variable that can be solved sequentially. These produce a search tree of solutions that we call *MQ tree*. We solve the original underdetermined MQ problem by traversing the MQ tree, solving a system of (blue) *linearised* equations, and eventually checking the consistency of the solution against k (magenta) equations.

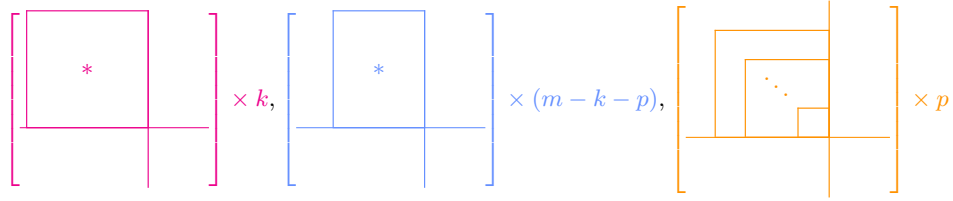


Fig. 4: Visualisation of the polynomials after our transformation.

Endpoints. Our novel algorithm has natural endpoints, see Figure 5. If we are in the massively underdetermined regime $n \geq m(m+1)/2$, then our approach allows for the choice $(k, p) = (0, m)$, thereby attaining the polynomial complexity of Miura, Hashimoto and Takagi [MHT13], creating a transformed shape identical to Figure 2. If we are in the hard regime where n is slightly larger than m , our optimization chooses $p = 0$ and $k \approx m - \sqrt{m}$, thereby reducing to an exhaustive search only algorithm. In the moderately underdetermined regime, which is used for signature constructions, our flexibility in choosing k and p is especially useful when the *underdeterminedness rate*, which we define as $\rho = \frac{2n}{m^2+m}$, is large. For example, for $m = 78$, one observes from Figure 5 that the cut-off point happens close to $n \approx 860$, i.e. for $\rho \approx 0.28$ utilised by MAYO [BCC⁺24]. Notice that for rate $\rho \geq 1$ we have $n \geq m(m+1)/2$, allowing our algorithm to enter the polynomial time regime of Miura, Hashimoto and Takagi [MHT13]. On the other hand, Hashimoto’s two blocks algorithm being more rigid never enters the polynomial time regime.

The simplicity and flexibility of our algorithm permits a clean asymptotic analysis: by optimizing the parameters (k, p) , one obtains the running time

$$\tilde{O}(q^{m-\sqrt{2n-m}}).$$

This closed asymptotic formula neatly interpolates between the polynomial endpoint and the exhaustive-search endpoint described above.

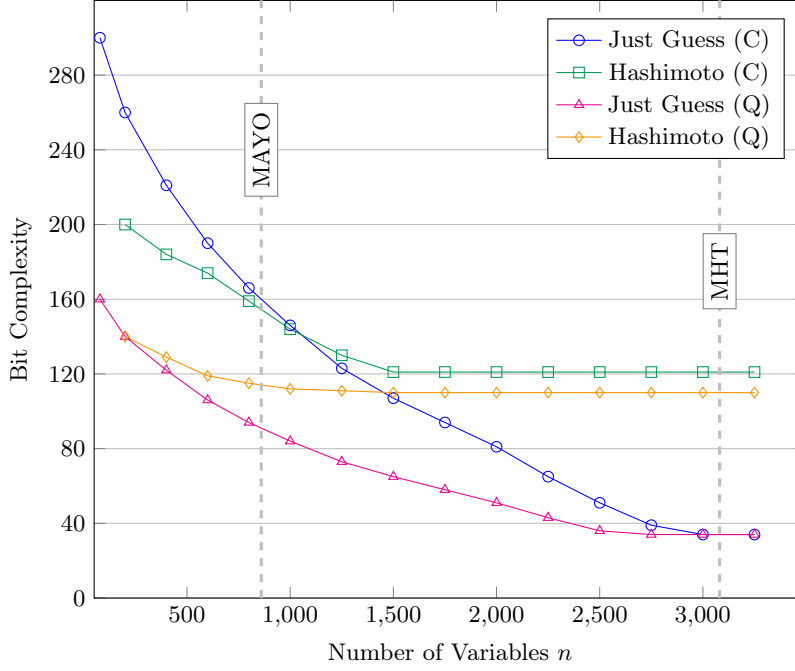


Fig. 5: Comparison for fixed $m = 78$, $q = 16$ and an increasing number of variables. The gray dashed lines mark u-rates: $\rho = 0.28$ of MAYO [BCC⁺24] security level I and $\rho = 1$ for the polynomial time regime of [MHT13].

Comparison with Hashimoto. Whereas in Hashimoto’s two-block strategy the change of variables permits the *linearization* of a single quadratic equation, our more flexible strategy leads to the linearization of multiple quadratic equations. Improved linearization is beneficial, since it reduces both the number of variables and equations in an MQ problem, making it significantly easier to solve. These special equations correspond to the $m - k - p$ (blue) matrices in Figure 4.

More Guessing. As in Hashimoto’s transformation, we also enhance our transformation with the guessing strategy by Furue, Nakamura and Takagi [FNT21].

Differently to Hashimoto’s hybrid, our novel approach pushes to the extreme this shift of workload: on a high level, whereas Hashimoto finishes with a structure as in Figure 3, where the MQ workload is balanced among two large blocks, our strategy as visualized in Figure 4 allows to obtain the extreme structure with p (yellow) quadratic equations in one single variable, which can be solved using a look-up table, and no expensive Gröbner basis algorithm.

When ρ is sufficiently large, the flexibility of our attack allows for improved concrete classical complexities compared to Hashimoto’s hybrid. As an example, we included in Table 1 a parameter set MAYO (old) with underdeterminedness

rate $\rho = 0.41$ that was originally proposed by Beullens in [Beu22], where our algorithm outperforms Hashimoto’s hybrid by 10 bits.

Table 1: Our results for the Level I of NIST Round 2 signature schemes in comparison to Hashimoto’s attack [Has21].

Scheme	ρ	Classical		Quantum	
		Hashimoto	This work	Hashimoto	This work
MAYO	0.28	155	158	114	90
MAYO (old)	0.41	116	106	97	64
QR-UOV	0.17	182	183	119	103
SNOVA	0.14	187	218	127	120

Quantum Speedups. Classically, our runtime complexities in Table 1 are comparable to Hashimoto’s. Except for few similarities, the flavour of our attack is completely different from that of Hashimoto. Whereas after Hashimoto’s transformation one still has to solve a hard MQ problem via Gröbner basis computation, after our transformation one has to face a purely combinatorial exhaustive search problem.

While such a shift away from MQ may already be undesirable for an MQ based scheme, it can still be considered non-critical as long as the total complexity remains infeasible. Quantumly, the situation appears to be different, since hard MQ problems are so far highly resistant to quantum speedups, whereas exhaustive search benefits from Grover search’s quadratic speedup [Gro98]. This implies that our attack with an extreme focus on guessing fully enjoys the quantum quadratic speedup, when simply replacing classical exhaustive search by quantum Grover search.

From Table 1, we observe significant concrete quantum complexity improvements for the Level I parameter sets of all NIST Round 2 MQ signature schemes. For MAYO we improve by 24 bits, for QR-UOV by 16 bits, and for SNOVA by 7 bits.

Implementation. We have implemented our algorithm in SageMath. The implementation, the optimization script and a script that evaluates the complexity of our algorithm can be found in the following git repository https://github.com/maxostuzzi/underdefinedMQ_implementation.

Organization. In Section 2, we provide some useful notions for our novel solver of underdetermined MQ problems. In Section 3, we give a high level description of our JUST-GUESS SOLVER, together with a small example. In Section 4, we prove the correctness of JUST-GUESS SOLVER and analyse its asymptotic and concrete complexity. The technical details of our novel transformation are given in Section 5. Finally, Section 6 contains the results of applying our signature forgery attack to MAYO, QR-UOV and SNOVA.

2 Preliminaries

Matrix Form. Throughout the paper, we consider without loss of generality only *homogeneous* quadratic polynomials, i.e. every monomial has exactly degree two. This allows to write the polynomial as an upper-triangular matrix. Let $p(\mathbf{x}) = \sum_{1 \leq i \leq j \leq n} p_{ij} x_i x_j$ be such a polynomial. We associate to p the matrix $P = (p_{ij})_{1 \leq i \leq j \leq n}$, where the non-specified entries are considered to be zero. Conversely, from an upper-triangular matrix we can recover the corresponding polynomial in $\mathbf{x}$ as $p(\mathbf{x}) = \mathbf{x}^t P \mathbf{x}$. We refer interchangeably to quadratic polynomials $p(\mathbf{x})$ and their corresponding matrix representation P .

Multivariate Quadratic Maps. Let us fix positive integers $n, m, q \in \mathbb{Z}$, with q a prime power. A *multivariate quadratic map* (MQ map) is a function $P : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$ such that

$$P(\mathbf{x}) = \begin{bmatrix} p_1(\mathbf{x}) \\ \vdots \\ p_m(\mathbf{x}) \end{bmatrix},$$

where each $p_i(\mathbf{x})$ is a quadratic polynomial in the n variables $\mathbf{x} = (x_1, \dots, x_n)^t$. We are going to denote by $P^{(i)}$ the upper-triangular matrix associated with the polynomial $p_i(\mathbf{x})$, and by $P_{j,k}^{(i)}$ its entry at row j and column k .

Given an MQ map P and a *target* $\mathbf{t} \in \mathbb{F}_q^m$, the MQ problem $MQ(n, m)$ with n variables and m equations is to find on input $(P, \mathbf{t})$ a solution $\mathbf{x} \in \mathbb{F}_q^n$ satisfying $P(\mathbf{x}) = \mathbf{t}$. We denote by $GB(n, m)$ the complexity of solving an MQ problem $MQ(n, m)$ with n equations and m variables, where the field $\mathbb{F}_q$ is clear from the context.

Definition 1 (u-rate). We say that $MQ(n, m)$ is square, overdetermined or underdetermined if $n = m$, $n < m$ or $n > m$, respectively. In the underdetermined case, we define the u-rate (short for underdeterminedness rate) as $\rho = \frac{2n}{m^2 + m}$.

Block Matrix Convention. To better convey the ideas of our algorithms, we often resort to visual representations of polynomials as block matrices. Blocks labeled with an integer k denote square blocks of size $k \times k$ of arbitrary entries. Blocks labeled with $*$ denote non-square blocks with arbitrary entries. Blocks labeled with $\mathbf{1}_k$ denote identity submatrices of size $k \times k$, meaning that all the entries are zero except for the diagonal, where the entries are 1. Blank parts of matrices are intended to be zero. Moreover, when depicting matrices associated to a quadratic polynomial, we only draw the top left square submatrix of size m . As an example, Figure 6 has to be intended as ℓ square upper-triangular matrices all of the same shape and size $n \times n$, even if we only draw the top left square submatrix of size m . All these matrices have an identity block of size k on the top left, a non-square block of zeros below the identity block, a non-square block of arbitrary entries on the top right and a square block of arbitrary entries of size k on the bottom right.

$$\begin{bmatrix} \boxed{\mathbf{1}_k} & \boxed{*} \\ & \boxed{m-k} \end{bmatrix} \times \ell,$$

Fig. 6: Example for our matrix convention.

3 High-Level Solver for Undetermined MQ Systems

Let us start by giving a high-level version of our JUST-GUESS-SOLVER (Algorithm 1) to solve underdetermined MQ(n, m) problems $(P, \mathbf{t})$. In addition to $(P, \mathbf{t})$, we define two parameters:

- $k \in \{0, \dots, m\}$, the number of *guessed coordinates*, and
- $p \in \{0, \dots, m-k\}$, the number of MQ(1, 1) *problems* to solve.

Algorithm 1: JUST-GUESS-SOLVER

Input: MQ(n, m) problem $(P, \mathbf{t})$ with $P : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$; k enumerated coordinates; $0 \leq p \leq m-k$ MQ(1, 1) problems to be solved.

Output: Solution $\mathbf{y} \in \mathbb{F}_q^n$ to $(P, \mathbf{t})$, or FAIL.

- 1 Compute the changes of variables S, T and $(\tilde{P}, \tilde{\mathbf{t}})$. // **Transformation phase**
- 2 Assign value $(0, \dots, 0) \in \mathbb{F}_q^{n-m}$ to the variables $x_{m+1}, \dots, x_n$.
// **Just Guess phase**
- 3 **for** all assignments $(u_{m-k+1}, \dots, u_m) \in \mathbb{F}_q^k$ to $x_{m-k+1}, \dots, x_m$ **do**
// **MQ Tree step**
 - 4 **if** all p MQ(1, 1) problem $(\tilde{P}^{(m)}, \tilde{\mathbf{t}}_m), \dots, (\tilde{P}^{(m-(p-1))}, \tilde{\mathbf{t}}_{m-(p-1)})$ are
consecutively solvable **then**
 - 5 Determine values $(u_m, \dots, u_{m-(p-1)})$ for $x_m, \dots, x_{m-(p-1)}$.
 - 6 Solve linear system $\tilde{P}^{(k+1)}, \dots, \tilde{P}^{(m-p)}$ to assign a value
 $(u_1, \dots, u_{m-k-p})$ for $x_1, \dots, x_{m-k-p}$. // **Linearization step**
 - 7 **if** $\mathbf{u} = (u_1, \dots, u_m, 0^{n-m})$ satisfies $\tilde{P}^{(1)}, \dots, \tilde{P}^{(k)}$. // **Check step**
 - 8 **then**
 - 9 **return** $\mathbf{y} = S\mathbf{u}$.
- 10 **return** FAIL.

Algorithm 1 is divided into the following two phases.

Transformation phase. Our goal is to compute a linear change of coordinates $S : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ and a linear system transformation $T : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ such

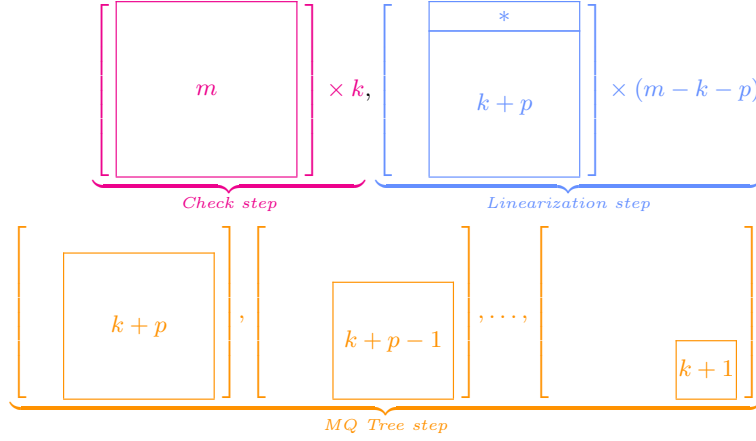


Fig. 7: Matrix representation of the polynomials of the transformed MQ map $\tilde{P}$.

that the transformed MQ problem $(\tilde{P}, \tilde{\mathbf{t}}) = (T \circ P \circ S, T\mathbf{t})$ is a sequence of $\text{MQ}(1, 1)$ problems that can be solved separately, up to some terms of the shape $x_i L_i(x_{m+1}, \dots, x_n)$, for $i \in \{1, \dots, m\}$ and linear maps L_i .

In line 2 of Algorithm 1, we assign to the variables $x_{m+1}, \dots, x_n$ the value 0^{n-m} . Recall that P is homogeneous of degree 2, and therefore $\tilde{P}$ as well. This implies that every $L_i(x_{m+1}, \dots, x_n)$ is homogeneous of degree 1 such that all linear maps L_i vanish at 0^{n-m} in each equation $1 \leq i \leq m$. Thus, we are left with a square $\text{MQ}(m, m)$ problem. Our transformation S, T ensures that the m equations $\tilde{P}^{(1)}, \dots, \tilde{P}^{(m)}$ in m variables achieve the structure depicted in Figure 7. The reason for this particular structure becomes clear in the Just Guess phase. We detail in Section 5 how to achieve this structure. In Section 5, we show that our transformation can be efficiently computed under the following conditions:

$$\begin{aligned} n-1 &\geq (m-k-p-1)(m-k+2), \\ n-m &\geq \frac{p(2m-2k-p-1)}{2}. \end{aligned} \tag{1}$$

Just Guess phase. We start by enumerating the q^k assignments for the variables $x_{m-k+1}, \dots, x_m$. For each assignment $(u_{m-k+1}, \dots, u_m) \in \mathbb{F}_q^k$, we solve p $\text{MQ}(1, 1)$ problems consecutively, starting from the last equation $\tilde{P}^{(m)}$.

MQ Tree step. In Figure 7, we observe that $\tilde{P}^{(m)}$ solely depends on the $k+1$ variables $x_{m-k}, \dots, x_m$, of which we have already fixed all but x_{m-k} . Thus, solving a $\text{MQ}(1, 1)$ problem yields a value u_{m-k} for x_{m-k} . In Section 4, we detail how to handle the issue that $\text{MQ}(1, 1)$ problems over $\mathbb{F}_q$ admit 0, 1, 2 or

q solutions, which results in traversing a solution tree. For the moment, assume we obtain a unique solution u_{m-k} .

Notice that $\tilde{P}^{(m-1)}$ depends on the $k+2$ variables $x_{m-k-1}, \dots, x_m$, which we have already fixed, except for x_{m-k-1} . Again, this allows us to solve an MQ(1, 1) problem in x_{m-k-1} , and so forth.

In total, the *MQ Tree step* from Algorithm 1 solves p MQ(1, 1) problems, corresponding to the p (yellow) matrices in staircase structure from Figure 7. This assigns a value to p more variables, leaving us with the still free variables $x_1, \dots, x_{m-k-p}$.

Linearization step. For determining the free variables we use the $m-k-p$ (blue) matrices $\tilde{P}^{(k+1)}, \dots, \tilde{P}^{(m-p)}$ from Figure 7. By the structure of these matrices, each matrix provides a linear equation in the free variables $x_1, \dots, x_{m-k-p}$. Thus, via Gaussian elimination we obtain (at least) one assignment for our free variables, thereby fixing all of them. Let us briefly explain why the blue matrices yield linear equations. Let $\mathbf{x} = (x_1, \dots, x_{m-k-p})^t$ be the free variables and $\mathbf{u} = (u_{m-k-p+1}, \dots, u_m)^t$ be the assignment for the variables $x_{m-k-p+1}, \dots, x_m$. In Figure 8, we observe that the block of size $k+p$ only contributes as a constant, while the rectangular block produces linear terms in $\mathbf{x}$.

$$\begin{bmatrix} \mathbf{x}^t & \mathbf{u}^t \end{bmatrix} \begin{bmatrix} * \\ k+p \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{u} \end{bmatrix} = \mathbf{u}^t \underbrace{\begin{bmatrix} k+p \end{bmatrix}}_{\text{Constant}} + \mathbf{x}^t \underbrace{\begin{bmatrix} * \end{bmatrix}}_{\text{Linear}} \mathbf{u}$$

Fig. 8: The blue matrices produce linear polynomials after the assignment of the variables $x_{m-k-p+1}, \dots, x_m$.

Check step. Notice that the *Just Guess phase* so far did not use the k (magenta) polynomials $\tilde{P}^{(1)}, \dots, \tilde{P}^{(k)}$ from Figure 7. Hence, in the *Check step* we have to verify whether our assignment $(u_1, \dots, u_n)$ also fulfills $\tilde{P}^{(1)}, \dots, \tilde{P}^{(k)}$, which happens with probability q^{-k} for random polynomials over $\mathbb{F}_q$. If this is not the case, one iterates the Just Guess phase with the subsequent assignment for the k variables $x_{m-k+1}, \dots, x_m$. Notice that JUST-GUESS-SOLVER succeeds if it finds among all q^k assignments a solution $\mathbf{u} \in \mathbb{F}_q^n$, otherwise it outputs FAIL. In Section 4.2 we analyze the success probability, while in Section 4.3 we show how to turn JUST-GUESS-SOLVER via rerandomization into an algorithm that always succeeds.

A solution $\mathbf{u} \in \mathbb{F}_q^n$ to the transformed MQ problem $(\tilde{P}, \tilde{\mathbf{t}})$ eventually yields a solution $\mathbf{y} = S\mathbf{u}$ to the original MQ problem $(P, \mathbf{t})$.

3.1 A Toy Example for JUST-GUESS-SOLVER

To make the reader familiar with the strategy from JUST-GUESS-SOLVER, let us provide an illustrative example. Assume we want to solve an MQ problem with input $(P, \mathbf{t})$ over $\mathbb{F}_3$, where $P : \mathbb{F}_3^{11} \rightarrow \mathbb{F}_3^6$, i.e. $n = 11$, $m = 6$ and $q = 3$. In this example, we choose $(k, p) = (2, 2)$, meaning that we enumerate two variables and we have to solve two MQ(1, 1) problems.

Figure 9 illustrates how JUST-GUESS-SOLVER proceeds for our toy example.

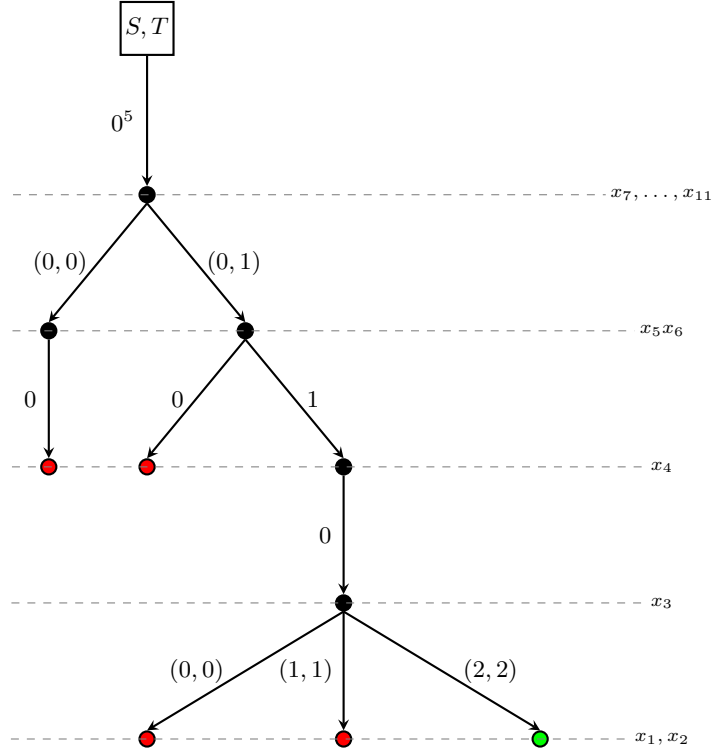


Fig. 9: Assignments of JUST-GUESS-SOLVER for the toy example.

Transformation Phase. Let us assume that the change of variables S , system transformation T and the transformed MQ problem $(\tilde{P}, \tilde{\mathbf{t}})$ have been computed. We assign the value $(u_7, \dots, u_{11}) = (0, \dots, 0)$ to the variables $x_7, \dots, x_{11}$. Therefore, we can omit the last five columns in $\tilde{P}^{(1)}, \dots, \tilde{P}^{(6)}$. Moreover, since the $P^{(i)}$ are upper triangular, we can also omit their last five rows. This leaves us with 6×6 matrices, as depicted in Figure 10. Let us further assume that the transformed target is $\tilde{\mathbf{t}} = (2, 0, 2, 1, 2, 0) \in \mathbb{F}_3^6$.

$$\begin{aligned}
\tilde{P}^{(1)} &= \begin{bmatrix} 1 & 1 & 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, & \tilde{P}^{(2)} &= \begin{bmatrix} 2 & 2 & 0 & 0 & 0 & 1 \\ 0 & 0 & 2 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 2 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{bmatrix} \\
\tilde{P}^{(3)} &= \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 0 & 2 \\ 0 & 0 & 1 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 2 \end{bmatrix}, & \tilde{P}^{(4)} &= \begin{bmatrix} 0 & 0 & 1 & 0 & 2 & 2 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\
\tilde{P}^{(5)} &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, & \tilde{P}^{(6)} &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 2 \\ 0 & 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}
\end{aligned}$$

Fig. 10: Matrices of the transformed MQ map $\tilde{P}$.

Just Guess phase. Let us write down in Figure 11 the equations corresponding to each matrix of $\tilde{P}$. Recall that the polynomials can be recovered from the matrices by computing $\tilde{p}_i(\mathbf{x}) = \mathbf{x}^t \tilde{P}^{(i)} \mathbf{x}$, where $\mathbf{x} = (x_1, \dots, x_6)^t$.

MQ Tree step. We enumerate the $k = 2$ variables x_5, x_6 in lexicographic order, starting with the assignment $(u_5, u_6) = (0, 0) \in \mathbb{F}_3^2$. The MQ Tree step has to solve $p = 2$ MQ(1, 1) problems. First, we evaluate $\tilde{P}^{(6)}$ at $(u_5, u_6) = (0, 0)$, obtaining $x_4^2 = 0$, which is fulfilled by the assignment $u_4 = 0$. Second, we evaluate $\tilde{P}^{(5)}$ at $(u_4, u_5, u_6) = (0, 0, 0)$, which yields the contradiction $0 = 2$.

Thus, we proceed with the next assignment $(u_5, u_6) = (0, 1) \in \mathbb{F}_3^2$. Evaluating $\tilde{P}^{(6)}$ yields $x_4^2 + 2x_4 = 0$, which is fulfilled by the two assignments $u_4 \in \{0, 1\}$. Evaluating $\tilde{P}^{(5)}$ at $(u_4, u_5, u_6) = (0, 0, 1)$ yields the contradiction $1 = 2$. However, evaluating $\tilde{P}^{(5)}$ at the alternative assignment $(u_4, u_5, u_6) = (1, 0, 1)$ gives $2x_3 = 0$, which is satisfied for $u_3 = 0$.

Linearization step. Evaluating $\tilde{P}^{(3)}, \tilde{P}^{(4)}$ at $(u_3, u_4, u_5, u_6) = (0, 1, 0, 1)$ leads to the linear system of equations

$$\begin{cases} x_1 + 2x_2 = 0 \\ 2x_1 + x_2 = 0 \end{cases}$$

This linear system has rank one, thus we have the $q = 3$ possible assignments $(u_1, u_2) \in \{(0, 0), (1, 1), (2, 2)\}$ for the variables x_1, x_2 . The three candidate as-

$$\begin{aligned}
\tilde{P}^{(1)} &= x_1^2 + x_1x_2 + 2x_1x_5 + 2x_2^2 + x_2x_6 + x_3x_4 + x_4^2 + x_5^2 + x_6^2 = 2 \\
\tilde{P}^{(2)} &= 2x_1^2 + 2x_1x_2 + x_1x_6 + 2x_2x_3 + x_2x_4 + x_3^2 + 2x_3x_6 + 2x_4^2 + x_5^2 + 2x_6^2 = 0 \\
\tilde{P}^{(3)} &= x_1x_5 + x_1x_6 + 2x_2x_3 + 2x_2x_6 + x_3^2 + 2x_3x_4 + x_4x_5 + 2x_5x_6 + 2x_6^2 = 2 \\
\tilde{P}^{(4)} &= x_1x_3 + 2x_1x_5 + 2x_1x_6 + x_2x_3 + x_2x_4 + 2x_3^2 + x_3x_5 + 2x_4x_5 + x_5x_6 + x_6^2 = 1 \\
\tilde{P}^{(5)} &= 2x_3x_4 + x_4^2 + 2x_5x_6 + x_6^2 = 2 \\
\tilde{P}^{(6)} &= x_4^2 + 2x_4x_6 + 2x_5^2 = 0
\end{aligned}$$

Fig. 11: The transformed MQ problem $(\tilde{P}, \tilde{t})$ in form of equations, after the assignments $(0, \dots, 0) \in \mathbb{F}_3^5$ to $x_7, \dots, x_{11}$.

signments

$$\begin{aligned}
\mathbf{u}_0 &= (0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0) \in \mathbb{F}_3^{11} \\
\mathbf{u}_1 &= (1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0) \in \mathbb{F}_3^{11} \\
\mathbf{u}_2 &= (2, 2, 0, 1, 0, 1, 0, 0, 0, 0, 0) \in \mathbb{F}_3^{11}
\end{aligned}$$

fulfill all equations $\tilde{P}^{(3)}, \dots, \tilde{P}^{(6)}$.

Check step. We verify that $\mathbf{u}_0, \mathbf{u}_1$ do not satisfy $\tilde{P}^{(1)}, \tilde{P}^{(2)}$. However, $\mathbf{u}_2$ satisfies both $\tilde{P}^{(1)}, \tilde{P}^{(2)}$ and therefore is a solution to the transformed MQ problem $(\tilde{P}, \tilde{t})$. The corresponding solution to the original MQ(n, m) problem (P, t) is $S\mathbf{u}_2$.

4 Complexity Analysis

Our JUST-GUESS-SOLVER (Algorithm 1) consists of two phases, the Transformation phase and the Just Guess phase. As we will see in Section 5, the Transformation phase runs in polynomial time. Thus, JUST-GUESS-SOLVER's complexity is dominated by the complexity of the Just Guess phase.

The Just Guess phase starts with the MQ tree step, which we analyze in the following. Recall from Section 3 and Figure 9 that the MQ tree step consecutively computes p solutions to MQ(1, 1) problems over $\mathbb{F}_q$. Since MQ(1, 1) problems admit 0, 1, 2, or q solutions, this leads to a search tree of assignments that we call *MQ tree*.

Definition 2 (MQ Tree). *Given a sequence of p consecutive MQ(1, 1) problems as in the Just Guess phase, we define an MQ tree as follows.*

1. The root node on level 0 corresponds to an assignment $u_{m-k+1}, \dots, u_m$ for the k variables $x_{m-k+1}, \dots, x_m$.
2. Every node $n(i)$ on level $0 \leq i < p$ has $\ell \in \{0, 1, 2, q\}$ children, where ℓ is the number of solutions to $(\tilde{P}^{(m-i)}, \tilde{t}_{m-i})$ with the current assignment. Let $u_1, \dots, u_\ell \in \mathbb{F}_q$ be these solutions. Then $n(i)$ has ℓ children corresponding to the assignments $u_1, \dots, u_\ell$.

Remark 1. The if-condition in line 4 of JUST-GUESS-SOLVER is satisfied if and only if the MQ tree with k -variable assignment $u_{m-k+1}, \dots, u_m$ has depth p .

Our JUST GUESS phase traverses an MQ tree via a standard Depth First Search.

In the following, we compute the number of nodes at depth p in an MQ tree for a fixed assignment $u_{m-k+1}, \dots, u_m$. To this end, we make the following heuristic assumption that is considered a mild standard assumption in the analysis of MQ solving algorithms.

Assumption 1 *Fixed assignments lead to independent random MQ problems. In particular, we need that for every node on level $0 \leq i < p$ in an MQ tree: the number $\ell \in \{0, 1, 2, q\}$ of solutions is distributed as in a random MQ(1, 1) problem.*

We experimentally verify the validity of Assumption 1 in Section 4.2. To better understand the structure of our MQ trees, in Section 4.1 we explicitly compute the probability that a single quadratic equation over $\mathbb{F}_q$ has zero, one, two or q solutions.

4.1 Number of Solutions of a Quadratic Equation

Let us denote by $\mathbf{P}_\ell$ the probability that a random MQ(1, 1) problem admits $\ell \in \{0, 1, 2, q\}$ solutions. Given $a, b, c \in \mathbb{F}_q$, let us consider the quadratic equation

$$ax^2 + bx + c = 0. \tag{2}$$

The coefficient a is zero with probability $1/q$. In this case, the equation admits no solution if $b = 0$ and $c \neq 0$; q distinct solutions if $b = 0$ and $c = 0$; one solution if $b \neq 0$.

Now, we need to differentiate between the even and odd characteristic case.

Even Characteristic. The coefficient a is non-zero with probability $\frac{q-1}{q}$. In this case, we can assume $a = 1$ without loss of generality. The equation admits one solution if additionally $b = 0$, since the squaring map is an isomorphism in even characteristic; two distinct solutions with probability $\frac{1}{2}$, if $b \neq 0$; no solution with probability $\frac{1}{2}$ if $b \neq 0$.

Odd Characteristic. Again, the coefficient a is non-zero with probability $\frac{q-1}{q}$ and we assume $a = 1$ without loss of generality. The discriminant of the equation is $\Delta = b^2 - 4c$, which is uniformly distributed provided b and c are uniformly distributed. The equation admits one solution if $\Delta = 0$, which happens with probability $1/q$; two distinct solutions if Δ is a square modulo q , which happens with probability approximately $1/2$; no solution if Δ is not a square modulo q , which happens with probability approximately $1/2$.

Lemma 1. *Let $a, b, c \in \mathbb{F}_q$ be sampled uniformly at random. Then, independently from the characteristic of the field $\mathbb{F}_q$, we have*

$$\begin{aligned} \mathbf{P}_0 &= \frac{(q-1)(q^2 - q + 2)}{2q^3}, & \mathbf{P}_1 &= \frac{2(q-1)}{q^2}, \\ \mathbf{P}_2 &= \frac{(q-1)^2}{2q^2}, & \mathbf{P}_q &= \frac{1}{q^3}. \end{aligned}$$

Moreover, the expected number of solutions of Equation (2) is 1.

4.2 Analysis of the Just Guess Phase

After the Transformation phase, our JUST-GUESS-SOLVER leaves us with an MQ(m, m) problem. During the Just Guess phase, we enumerate over q^k assignments of k variables, thereby addressing MQ($m - k, m$) problems. Every of these MQ($m - k, m$) problems leads to a solution that passes the Check step with probability q^{-k} .

However, we only enter the Check step when we succeed to solve all p MQ(1, 1) problems in line 4 of JUST-GUESS-SOLVER. By Remark 1 this happens if and only if the corresponding MQ tree T of this assignment has depth p . Let N_j denote the number of nodes of T at depth j . Then, the number of candidate solutions we obtain for each k -variable assignment is N_p , and the number of MQ(1, 1) problems solved on average in each MQ tree is $E[N_1 + \dots + N_p]$.

Lemma 2. *Let T be a tree in which each node has ℓ children with probability $\mathbf{P}_\ell$, for $\ell \in \{0, 1, 2, q\}$ as computed in Lemma 1. Then, the expected number of nodes at depth j in T is $E[N_j] = 1$ for all $j \geq 0$.*

Proof. For $j \in \{0, \dots, p\}$, let N_j be the number of nodes in the tree T at depth j . We have $E[N_0] = 1$ and $E[N_1] = \mathbf{P}_1 + 2\mathbf{P}_2 + q\mathbf{P}_q = 1$ by Lemma 1. By the law of total expectation, observe that

$$E[N_j] = E[E[N_j | N_{j-1}]] = E[E[N_1] N_{j-1}] = E[N_{j-1}].$$

By induction, this implies that $E[N_j] = 1$ for all $j \geq 0$. $\square$

Remark 2. By linearity of the expectation, the expected number of nodes in the tree T defined in Lemma 2 from depth one to depth p is

$$E \left[\sum_{j=1}^p N_j \right] = \sum_{j=1}^p E[N_j] = p.$$

Whereas Lemma 2 and Remark 2 refer to generic trees that have a certain number ℓ of children with some probability $\mathbf{P}_\ell$, under Assumption 1 MQ trees behave identically to these generic trees.

We have generated random MQ trees and successively solved MQ(1, 1) problems just as in JUST-GUESS-SOLVER, counting the number of full assignments of variables yielded and the number of MQ(1, 1) processed. The results are in Table 2 for $q \in \{7, 16\}$, the field sizes used in MAYO, QR-UOV and SNOVA.

Table 2: Number of nodes at depth p and number of MQ(1, 1) problems solved per real MQ trees, averaged over 10^4 trials for $q \in \{7, 16\}$.

p	$E[N_p]$		#MQ(1, 1)	
	$q = 7$	$q = 16$	$q = 7$	$q = 16$
1	1.00	1.00	1.00	1.00
10	1.03	1.01	9.83	9.98
20	0.96	0.97	19.70	19.66
30	1.02	1.00	30.04	29.06
40	0.97	0.99	39.99	40.13

Remark 3. Under Assumption 1, Lemma 2 implies that each assignment of the k variables $x_{m-k+1}, \dots, x_m$ produces on average one full assignment of variables that enters the Check step. For each of these, the probability to successfully pass the Check step is q^{-k} ¹.

Theorem 1. *Under Assumption 1, the success probability of the Just Guess phase is*

$$1 - 1/e + O(q^{-k}).$$

Proof. Just Guess phase succeeds if at least one of the assignments for the k variables $x_{m-k+1}, \dots, x_m$ successfully passes the MQ tree, Linearization and Check steps, which happens with probability q^{-k} by Remark 3. Therefore, the success probability of Just Guess phase is

$$P_{\text{success}} = 1 - (1 - q^{-k})^{q^k}.$$

After taking the natural logarithm, consider the Taylor expansion

$$\gamma = q^k \ln(1 - q^{-k}) = -1 - \frac{1}{2}q^{-k} + O(q^{-2k}).$$

¹ This holds assuming that the Linearization step produces one solution. We computed the probability in the general case, where obtaining zero or more than one assignment is possible. This significantly complicated the formulas, producing an inappreciable numerical improvement. We opted to omit this analysis, thereby slightly overestimating the run time complexity.

Therefore, by expanding e^γ , we have

$$P_{\text{success}} = 1 - e^{-\gamma} = 1 - 1/e + O(q^{-k}).$$

□

4.3 Rerandomisation

We turn JUST-GUESS-SOLVER into an algorithm that always succeeds by rerandomising our initial MQ problem, as described in FULL-JUST-GUESS-SOLVER (Algorithm 2).

Algorithm 2: FULL-JUST-GUESS-SOLVER

Input: MQ(n, m) problem $(P, \mathbf{t})$ with $P : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$
Output: Solution $\mathbf{y} \in \mathbb{F}_q^n$ to $(P, \mathbf{t})$, or FAIL.
1 Choose $0 \leq k \leq m$ for the number of guessed coordinates.
2 Choose $0 \leq p \leq m - k$ for the number of MQ(1, 1) problems.
3 **repeat**
4 Choose a random invertible linear change of variables $M \in \mathbb{F}_q^{n \times n}$.
5 $\mathbf{y}_M \leftarrow \text{JUST-GUESS-SOLVER}((P \circ M, \mathbf{t}), k, p)$
6 **until** $\mathbf{y}_M \neq \text{FAIL}$
7 **return** $\mathbf{y} = M \cdot \mathbf{y}_M$

Correctness. FULL-JUST-GUESS-SOLVER (Algorithm 2) rerandomises to the MQ system $P \circ M$. This rerandomises all matrix representations $P^{(1)}, \dots, P^{(m)}$ to $M^t P^{(1)} M, \dots, M^t P^{(m)} M$. Assume we obtain a solution $\mathbf{y}_M$ for the rerandomised MQ problem $(P \circ M, \mathbf{t})$. Then

$$\mathbf{y}_M^t \cdot M^t P^{(i)} M \cdot \mathbf{y}_M = t_i \text{ for all } 1 \leq i \leq m.$$

It follows that $\mathbf{y} = M \cdot \mathbf{y}_M$ solves the original $(P, \mathbf{t})$.

Complexity. Let us now analyze the runtime of FULL-JUST-GUESS-SOLVER, both classically and quantumly.

Theorem 2 (Runtime). *Let $k \in \{0, \dots, m\}$ and $p \in \{0, \dots, m - k\}$ be parameters satisfying Equation (1). Let C be the average complexity for running JUST-GUESS-SOLVER's Just Guess phase for a single assignment. For any fixed $\varepsilon \in (0, 1]$, under Assumption 1, FULL-JUST-GUESS-SOLVE solves $(P, \mathbf{t})$ with probability $1 - \varepsilon$ in time*

$$C \cdot q^k \log(1/\varepsilon) \text{ classically, and} \\ C \cdot \sqrt{q^k \log(1/\varepsilon)} \text{ quantumly.}$$

Moreover, *FULL-JUST-GUESS-SOLVER* finds one solution to the MQ problem $(P, \mathbf{t})$ in expected time at most

$$\begin{aligned} &C \cdot q^k \text{ classically, and} \\ &C \cdot q^{k/2} \text{ quantumly.} \end{aligned}$$

Proof. As already mentioned, the running time of *JUST-GUESS-SOLVER* is dominated by the Just Guess phase, since the Transformation phase runs in polynomial time.

For a fixed k -variable assignment in the Just-Guess phase, let us call *iteration* the MQ tree, Linearisation and Check steps together. After x iterations, the success probability of *FULL-JUST-GUESS-SOLVER* is $P_{\text{success}} = 1 - \varepsilon$, where $\varepsilon = (1 - q^{-k})^x$. We thus obtain

$$x = \frac{\log(\varepsilon)}{\log(1 - q^{-k})} \approx q^k \log(1/\varepsilon).$$

By replacing the classical enumeration by Grover's search, we obtain the quantum result.

We now count the expected number of iterations to find one solution. From Remark 3, we know that a single iteration yields a solution with probability q^{-k} . Therefore, in expectation, we have to run q^k iterations until we succeed, each of them of cost C . Again, classical enumeration can be replaced by a Grover search, and the complexity claims follow. $\square$

Remark 4 (Runtime Optimization). Since C is polynomial in n, m and q , Theorem 2 shows that *FULL-JUST-GUESS-SOLVER*'s runtime is minimized for the smallest possible k that satisfies Equation (1).

4.4 Asymptotic Complexity

As observed in Remark 4, the complexity of *FULL-JUST-GUESS-SOLVER* is classically $\tilde{O}(q^k)$ and quantumly $\tilde{O}(q^{k/2})$, where k is the smallest integer in $\{0, \dots, m\}$ such that there exists $p \in \{0, \dots, m-k\}$ such that the constraints in Equation (1) are fulfilled.

Theorem 3 (Asymptotic Complexity). *The asymptotic complexity of FULL-JUST-GUESS-SOLVER is*

$$\begin{aligned} &\tilde{O}(q^{m-\sqrt{2n-m}}) \text{ classically and} \\ &\tilde{O}(q^{\frac{1}{2}m-\frac{1}{2}\sqrt{2n-m}}) \text{ quantumly.} \end{aligned}$$

Proof. Let us denote $M = m-k$. Then, once $M \in \{0, \dots, m\}$ and $p \in \{0, \dots, M\}$ satisfying Equation (1) are fixed, the asymptotic complexity of *FULL-JUST-GUESS-SOLVER* is $\tilde{O}(q^{m-M})$ classically and $\tilde{O}(q^{\frac{1}{2}(m-M)})$ quantumly. Therefore, we need to find the maximal M such that there exists p satisfying Equation (1).

Rearranging the first constraint of Equation (1), we obtain the following lower bound on p

$$p \geq M - 1 - \frac{n}{M+2}.$$

Rearranging the second constraint Equation (1), we obtain the inequality

$$p^2 - (2M - 1)p + 2(n - m) \geq 0. \quad (3)$$

If the discriminant $(2M - 1)^2 - 8(n - m) < 0$ for a fixed M , then all choices for $p \in \{0, \dots, M\}$ are admissible. Solving for p the associated quadratic equation yields the roots

$$r_{\pm} = \frac{(2M - 1) \pm \sqrt{(2M - 1)^2 - 8(n - m)}}{2}.$$

Thus, from Equation (3) we obtain the upper bound $p \leq r_-$ and the lower bound $r_+ \leq p$ on p . However, notice that $p \leq M < r_+$, which makes the lower bound on p void. Therefore, the second constraint of Equation (1) ultimately yields only an upper bound. Combining the lower and the upper bounds on p and ignoring the additive constants we obtain

$$M - \frac{n}{M} \leq p \leq \frac{2M - \sqrt{(2M)^2 - 8(n - m)}}{2},$$

where we assume that $M \neq 0$, i.e. $k < m$. Hence, for a fixed choice of M , which we ultimately want to maximise, there exists a $p \in \{0, \dots, M\}$ satisfying Equation (1) if and only if

$$M - \frac{n}{M} \leq \frac{2M - \sqrt{4M^2 - 8(n - m)}}{2}.$$

For the moment, consider only the right side of the inequality: by multiplying and dividing by $2M + \sqrt{4M^2 - 8(n - m)}$, we obtain

$$\frac{8(n - m)}{2(2M + \sqrt{4M^2 - 8(n - m)})} \geq \frac{8(n - m)}{8M} = \frac{n - m}{M},$$

where the inequality follows from $\sqrt{4M^2 - 8(n - m)} \leq 2M$. The condition for the existence of p becomes

$$M - \frac{n}{M} \leq \frac{(n - m)}{M},$$

which yields $M \leq \sqrt{2n - m}$. In order to minimise the asymptotic complexity, we choose the largest M , i.e. $M = \sqrt{2n - m}$. The claim follows. $\square$

Our algorithm can be applied to square MQ problems as well, namely when $n = m$. In this edge case, the optimisation selects $p = 0$ and the smallest k satisfying the constraints in Equation (1). Indeed, in this case our algorithm still follows our guessing strategy and asymptotically outperforms naive enumeration's complexity of $\tilde{O}(q^n)$.

Corollary 1. *For square MQ(n, n) problems, the asymptotic complexity of FULL-JUST-GUESS-SOLVER is*

$$\tilde{O}(q^{n-\sqrt{n}}) \quad \text{and} \quad \tilde{O}(q^{\frac{1}{2}n - \frac{1}{2}\sqrt{n}})$$

classically and quantumly respectively.

4.5 Polynomial Overhead in the Just-Guess Phase

By Theorem 2 we obtain an exponential (in k) number of iterations, where each iteration costs polynomial overhead C .

The cost C splits in the expected costs of the three steps MQ Tree, Linearization and Check. Thus,

$$C = C_{\text{MQTree}} + E[N_p](C_{\text{Linearization}} + C_{\text{Check}}),$$

where $E[N_p] = 1$, from Lemma 2.

Expected Cost C_{Check} . Let c be the cost of verifying that a variable assignment fulfills one equation. Then, the expected cost of the Check step is

$$c \cdot \sum_{j=1}^k \left(\frac{1}{q}\right)^{j-1} = c \frac{q}{q-1} (1 - q^{-k}) \leq c \frac{q}{q-1},$$

for q^{-k} small.

Expected Cost C_{MQTree} . Let $C_{\text{assign}}(i)$ be the cost of converting one matrix into a polynomial in the MQ tree step after assigning $k+i$ variables. Therefore, each node at level i of the MQ tree costs $C_{\text{assign}}(i)$ for computing the polynomial and one step for solving the corresponding MQ(1,1) problem. From Lemma 2 and Remark 2, the expected number of nodes visited at any depth of the MQ tree is one, and the expected number of total MQ(1,1) problems processed in the MQ tree phase is then exactly p . Therefore, we have that the expected cost C_{tree} of the MQ tree step is in total

$$C_{\text{MQTree}} = p + \sum_{i=0}^p C_{\text{assign}}(i).$$

We used naive, non-optimized algorithms for performing assignments to polynomials, Linearization and Check, whose exact complexity formulas C_{assign} , $C_{\text{Linearization}}$, C_{Check} can be found in our git repository ².

5 Our Transformation of Underdetermined MQ Maps

In this section, we introduce our new algorithm to compute a change of variables $S : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ and a system transformation $T : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ for the MQ map P to allow the Just Guess phase. Our transformation is a generalisation of Miura, Hashimoto and Takagi [MHT13].

² https://github.com/maxostuzzi/underdefinedMQ_implementation

5.1 Computing the Transformation

We now describe the algorithm TRANSFORMATION, that on input an MQ map P and the parameters $k \in \{0, \dots, m\}$ and $p \in \{0, \dots, m - k\}$ outputs a linear change of coordinates $S : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ and a system transformation $T : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ allowing the Just Guess phase. TRANSFORMATION builds on two subroutines that we call ELIMINATEDIAGONAL and ELIMINATEABOVE. ELIMINATEDIAGONAL has essentially already been introduced in [MHT13]. ELIMINATEABOVE is inspired by a technique first described in [MHT13]. The way we combine the two algorithms to obtain the desired form depicted in Figure 7 is novel.

Generalised Gaussian Elimination. ELIMINATEDIAGONAL is essentially one step of Gaussian elimination generalised to MQ maps: taking linear combinations of the polynomials of P , the algorithm eliminates one square coefficient, i.e. a diagonal term, from a portion of the polynomials. More precisely, given a MQ map P , an *equation index* e and a *column index* i as input, ELIMINATEDIAGONAL outputs a linear change of coordinates $T : \mathbb{F}_q^m \rightarrow \mathbb{F}_q^m$ such that

$$\tilde{P}_{i,i}^{(\ell)} = 0, \quad \text{for } \ell \in \{e + 1, \dots, m\},$$

where $\tilde{P} = T \circ P$. The transformation T is obtained by adding to $P^{(\ell)}$ a suitable multiple of $P^{(e)}$, provided $P_{i,i}^{(e)}$ is nonzero. If $P_{i,i}^{(e)} = 0$, we reorder the last $m - e$ polynomials such that the e -th polynomial has a nonzero coefficient in the monomial x_ℓ^2 . If this coefficient is zero in all the last $m - e$ polynomials, there is no elimination to be done and the algorithm simply returns $T = \mathbf{1}_m$.

Eliminating Mixed Term Coefficients. The goal of ELIMINATEABOVE is to compute a linear change of coordinates $S : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$ such that a portion of the coefficients above the diagonal term of the matrices associated with $\tilde{P} = P \circ S$ is zero. In terms of polynomials, the change of coordinates computed by ELIMINATEABOVE eliminates part of the mixed terms $x_i x_j$ from $P^{(\ell)}$, where i is given as input and j, ℓ depend on i . We now describe ELIMINATEABOVE in more detail. The algorithm takes as input an MQ map P and a column index i . Consider the linear transformation associated with the matrix S in Figure 12, where $\mathbf{s}^t = (s_1, \dots, s_{i-1}, 1, s_{i+1}, \dots, s_n)$ is the i -th column. Let us define $\tilde{P} = P \circ S$. Note that in terms of the matrices associated with the MQ map we have $\tilde{P}^{(\ell)} = S^t P^{(\ell)} S$.

ELIMINATEABOVE operates in three different ways, depending on whether the column index input i lies in A_1, A_2 or A_3 , where

$$\begin{aligned} A_1 &= \{2, \dots, m - k - p\}, \\ A_2 &= \{m - k - p + 1, \dots, m - k\}, \\ A_3 &= \{m - k + 1, \dots, m\}. \end{aligned}$$

Note that $A_1 \cup A_2 \cup A_3 = \{2, \dots, m\}$ is a partition of the set of possible column indices excluding the first one, since the first column has no entry above the

$$S = \left[\begin{array}{c|c} \boxed{\mathbf{1}_{i-1}} & \begin{array}{c} i \\ \mathbf{s} \end{array} \\ \hline & \boxed{\mathbf{1}_{n-i}} \end{array} \right]$$

Fig. 12: Linear change of variables $S : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^n$.

diagonal. Indeed, our transformation operates column by column progressively, see Algorithm 3.

In the first case, when $i \in A_1$, ELIMINATEABOVE solves a system of linear equations to determine $\mathbf{s}$ such that $\tilde{P}^{(\ell)}$ matches the shape illustrated in Figure 13, for all $\ell \geq e_1(i)$, where $e_1(i) = 2k + p - m + i$. In words, for $\ell \geq e_1(i)$, the matrices $\tilde{P}^{(\ell)}$ have zeros above the diagonal of the i -th column. Note that the left side of the i -th row and the bottom side of the i -th column are zero as well, since all the matrices are upper-triangular in our notation. This condition is used in TRANSFORMATION (Algorithm 3) to linearise the $m - k - p$ blue equations of Figure 7. In order not to run into an undefined behaviour, we require the structural condition $2k + p - m \geq 0$ to be met, so that $e_1(i) \geq 2$ for all $i \in A_1$.

$$\left[\begin{array}{c|c} \boxed{i-1} & \begin{array}{c} i \\ 0 \\ \vdots \\ 0 \\ * \end{array} \\ \hline & \begin{array}{c} m \times (m-i) \end{array} \end{array} \right]$$

Fig. 13: Effect of ELIMINATEABOVE on the matrices $\tilde{P}^{(\ell)}$ for $e_1(i) \leq \ell \leq m$, when $i \in A_1$.

In the second case, when $i \in A_2$, the first $m - p$ (magenta and blue) matrices are not affected. ELIMINATEABOVE solves a system of linear equations to deter-

mine $\mathbf{s}$ such that two conditions are met: similarly to the previous case, $\tilde{P}^{(\ell)}$ must match the shape illustrated in Figure 13, for all $\ell \geq e_2(i)$ where $e_2(i) = k + i$. Additionally, the matrix $\tilde{P}^{(e_2(i)-j)}$ has $i - j - 1$ zeros in the i -th column, for $j \in \{1, \dots, i - (m - k - p + 1)\}$. These conditions are used in TRANSFORMATION (Algorithm 3) to clear the left blocks of the p yellow matrices in Figure 7.

In the last case, when $i \in A_3$, again the first $m - p$ (magenta and blue) matrices of Figure 7 are not affected. The condition is the following: the first $m - (k + p) + j$ entries of the i -th column of the matrix $\tilde{P}^{(m-p+j)}$ must be zero, for $j \in \{1, \dots, p\}$. These conditions are used in TRANSFORMATION (Algorithm 3) to create the staircase structure in the p yellow matrices of Figure 7.

The Full Transformation. Having described the two subroutines, the algorithm TRANSFORMATION consists of sequentially applying them column by column as detailed in Algorithm 3.

Algorithm 3: TRANSFORMATION

Input: MQ map P ; $k \in \{0, \dots, m\}$ and $p \in \{0, \dots, m - k\}$.
Output: Linear maps S and T .

```

1  $S \leftarrow \mathbf{1}_n$ 
2  $T \leftarrow \text{ELIMINATEDIAGONAL}(P, 2k + p - m + 1, 1)$ 
3  $\tilde{P} \leftarrow T \circ P$ 
4 for  $2 \leq i \leq m - k - p$  do
5      $S \leftarrow S \cdot \text{ELIMINATEABOVE}(\tilde{P}, i)$  // 1st case
6      $\tilde{P} \leftarrow T \circ P \circ S$ 
7      $T \leftarrow \text{ELIMINATEDIAGONAL}(\tilde{P}, 2k + p - m + i, i) \cdot T$ 
8      $\tilde{P} \leftarrow T \circ P \circ S$ 
9 for  $m - k - p < i \leq m - k$  do
10     $S \leftarrow S \cdot \text{ELIMINATEABOVE}(\tilde{P}, i)$  // 2nd case
11     $\tilde{P} \leftarrow T \circ P \circ S$ 
12     $T \leftarrow \text{ELIMINATEDIAGONAL}(\tilde{P}, k + i, i) \cdot T$ 
13     $\tilde{P} \leftarrow T \circ P \circ S$ 
14 for  $m - k < i \leq m$  do
15     $S \leftarrow S \cdot \text{ELIMINATEABOVE}(\tilde{P}, i)$  // 3rd case
16     $\tilde{P} \leftarrow T \circ P \circ S$ 
17 return  $S, T$ 
```

Note that, in general, ELIMINATEDIAGONAL and ELIMINATEABOVE do not commute. In other words, the order of application of the subroutines is crucial: inverting ELIMINATEABOVE and ELIMINATEDIAGONAL would not lead to the desired final form.

5.2 Parameter Requirements

ELIMINATEABOVE internally solves systems of linear equations. Assuming the randomness of such systems, we require that the number of equations does not exceed the number of variables in each of these linear systems to guarantee their solvability.

Every desired zero entry corresponds to one linear equation: we thus count the number of equations by counting the number of zeros we want to achieve. Let A_1, A_2, A_3 be the partition of $\{2, \dots, m\}$ defined in ???. We start with the first case of ELIMINATEABOVE, namely when $i \in A_1$. In particular, in the i -th iteration of the first **for**-cycle of Algorithm 3, ELIMINATEABOVE aims to obtain $i - 1$ zeros in $m_i = m - (e_1(i) - 1) = 2m - 2k - p - i + 1$ matrices and the number of variables available is $n - i$. Imposing the solvability condition and reindexing starting from one, we obtain the condition

$$n - 1 \geq i(2m - 2k - p - i + 1) \quad \text{for } 1 \leq i < m - k - p.$$

Since the left hand side is a quadratic function in i maximized for $i = m - k - p/2 + 1/2$, which is out of range, the most restrictive constraint is the one relative to $i = m - k - p - 1$, yielding

$$n - 1 \geq (m - k - p - 1)(m - k + 2).$$

Regarding the second and third cases of ELIMINATEABOVE, when $i \in A_2 \cup A_3$, the number of zeros is maximised in the last iteration of the third **for**-cycle of Algorithm 3. In this step, looking at the last column of the yellow matrices in Figure 7, we want to achieve $m - (k + 1)$ zeros in the last matrix, $m - (k + 2)$ in the $(m - 1)$ -th matrix, $m - (k + 3)$ in the previous one, and so forth, ending with $m - (k + p)$ zeros in the first of the yellow matrices. In total, the number of zeros is therefore $\sum_{i=1}^p m - k - i$, while the number of available variables is $n - m$. Imposing the solvability condition and rewriting, one obtains

$$n - m \geq \frac{p(2m - 2k - p - 1)}{2}.$$

In summary, the Transformation phase succeeds if the number of variables n and the number of equations m fulfill the following two requirements from Eq. (1)

$$\begin{aligned} n - 1 &\geq (m - k - p - 1)(m - k + 2), \\ n - m &\geq \frac{p(2m - 2k - p - 1)}{2}. \end{aligned}$$

6 Results for MAYO, QR-UOV and SNOVA

In this section, we provide estimates for the bit complexity of forging a signature for MAYO [Beu22,BCC⁺24], QR-UOV [FIKT21,FIH⁺24] and SNOVA

[WTKC22,WCD⁺24] with FULL-JUST-GUESS-SOLVER (Algorithm 2). We compare our results to the currently best known algorithm for solving underdetermined problems by Hashimoto [Has21].

The first green line for each parameter set corresponds to the estimates for Hashimoto’s algorithm. The bit complexity estimates for Hashimoto’s algorithm rely on the estimator [EVZB24].

For FULL-JUST-GUESS-SOLVER on input n, m, q we find the optimal parameters (k, p) that minimize the concrete classical and quantum runtime complexity using Theorem 2 and Remark 4. Our optimization code is available via our git repository ³. Since the field size q is small in all considered schemes, we attribute cost 1 to MQ(1,1) problems, by using lookup tables.

To get an impression of the polynomial overhead from Section 4.5, we also computed all runtimes without polynomial overhead (provided in parenthesis), i.e., by setting the polynomial overheads $C_{\text{assign}} = C_{\text{Linearization}} = C_{\text{Check}} = 0$. Taking the security level I parameters from MAYO, the bit complexity notion 158(141) means that 158 is the total bit complexity including 17 bits of polynomial overhead. Therefore, 141 bit is a lower bound for FULL-JUST-GUESS-SOLVER for any optimized implementation.

6.1 MAYO

MAYO [Beu22,BCC⁺24] is a relatively new digital signature in which forging is equivalent to solving an underdefined MQ problem. Among the schemes that we analyze, MAYO uses the largest u-rate ρ (Definition 1), which is beneficial for our transformation.

Round 2 Specification. For Table 3, we choose parameter sets from [BCC⁺24]. While classically Hashimoto’s algorithm still provides lower complexities, quantumly we obtain significant improvements for all security levels, by 24, 21 and 21 bits respectively.

Table 3: MAYO Round 2 [BCC⁺24] bit complexity estimates, in parenthesis w/o polynomial overhead. H stands for Hashimoto, best complexity is bold.

Security	(n, m, q)	ρ	k	p	Classical	Quantum
Level I	(860, 78, 16)	0.28	H		155	114
			34	25	158(141)	90 (73)
Level III	(1298, 108, 16)	0.22	H		222	153
			54	31	240(221)	132 (113)
Level V	(1848, 142, 16)	0.18	H		295	200
			77	37	333(313)	179 (159)

³ https://github.com/maxostuzzi/underdefinedMQ_implementation

Original MAYO paper. We also included the parameter sets originally proposed by Beullens [Beu22], see Table 4. These parameters use significantly larger u-rate ρ , which in turn leads to substantial improvements for our attack. Precisely, we are able to reduce the classical complexity for security level I and III by 10 and 2 bits respectively, and the quantum complexity for all security level by 34, 38 and 37 bits respectively.

Table 4: MAYO Original [Beu22] bit complexity estimates, in parenthesis w/o polynomial overhead. H stands for Hashimoto, best complexity is bold.

Security	(n, m, q)	ρ	k	p	Classical	Quantum
Level I	(924, 67, 16)	0.41	H		116	98
			21	26	106 (89)	64 (47)
Level III	(1666, 99, 16)	0.33	H		174	136
			37	36	172 (153)	98 (79)
Level V	(2470, 132, 16)	0.28	H		242	176
			57	44	253(233)	139 (119)

6.2 QR-UOV

QR-UOV [FIH⁺24] uses smaller u-rates ρ than MAYO, but also provides parameter sets with a small field $\mathbb{F}_7$ that compensates for the small ρ . Our results are shown in Table 5, where we improve over Hashimoto’s attack quantumly by 17, 19 and 24 bits.

Table 5: QR-UOV [FIH⁺24] bit complexity estimates, in parenthesis w/o polynomial overhead. H stands for Hashimoto, best complexity is bold.

Security	(n, m, q)	ρ	k	p	Classical	Quantum
Level I	(840, 100, 7)	0.17	H		182	120
			57	24	183(164)	103 (85)
Level III	(1240, 140, 7)	0.13	H		259	166
			88	30	271(252)	148 (128)
Level V	(1680, 190, 7)	0.09	H		351	231
			129	34	388(367)	207 (185)

6.3 SNOVA

Since SNOVA [WCD⁺24] has the smallest u-rate ρ among all NIST Round 2 candidates and field size $q = 16$, our quantum improvements are smallest in this setting. Still, we gain 8, 5 and 3 bits.

Table 6: SNOVA [WCD⁺24] bit complexity estimates, in parenthesis w/o polynomial overhead. H stands for Hashimoto, best complexity is bold.

Security	(n, m, q)	ρ	k	p	Classical	Quantum
Level I	(464, 80, 16)	0.14	H 49	17	185 218(200)	128 120 (102)
Level III	(725, 125, 16)	0.09	H 86	21	282 368(348)	201 196 (176)
Level V	(1120, 160, 16)	0.09	H 111	27	353 469(449)	250 247 (227)

Acknowledgments. Alexander May is supported by DFG under Germany’s Excellence Strategy - EXC 2092 CASA - 390781972. Massimo Ostuzzi is part of the Quantum-Safe Internet (QSI) ITN, which received funding from the European Union’s Horizon-Europe programme as Marie Skłodowska-Curie Action (PROJECT 101072637 - HORIZON - MSCA-2021-DN-01). Massimo Ostuzzi is supported by the BMBF through project Quantum and Benchmarks for Resource Allocation (QuBRA).

References

- BCC⁺24. Ward Beullens, Fabio Campos, Sofia Celi, Basil Hess, and Matthias J. Kannwischer. MAYO. Technical report, National Institute of Standards and Technology, 2024. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- Beu22. Ward Beullens. MAYO: Practical post-quantum signatures from oil-and-vinegar maps. In Riham AlTawy and Andreas Hülsing, editors, *SAC 2021*, volume 13203 of *LNCS*, pages 355–376. Springer, Cham, September / October 2022.
- BG94. Mihir Bellare and Shafi Goldwasser. The complexity of decision versus search. *SIAM Journal on Computing*, 23(1):97–119, 1994.
- CHMT14. Chen-Mou Cheng, Yasufumi Hashimoto, Hiroyuki Miura, and Tsuyoshi Takagi. A polynomial-time algorithm for solving a class of underdetermined multivariate quadratic equations over fields of odd characteristics. In Michele Mosca, editor, *Post-Quantum Cryptography - 6th International Workshop, PQCrypto 2014*, pages 40–58. Springer, Cham, October 2014.
- EVZB24. Andre Esser, Javier A. Verbel, Floyd Zweydinger, and Emanuele Bellini. {SoK: CryptographicEstimators - a Software Library for Cryptographic Hardness Estimation}. In *AsiaCCS*. {ACM}, 2024.
- FIH⁺24. Hiroki Furue, Yasuhiko Ikematsu, Fumitaka Hoshino, Tsuyoshi Takagi, Kan Yasuda, Toshiyuki Miyazawa, Tsunekazu Saito, and Akira Nagai. QR-UOV. Technical report, National Institute of Standards and Technology, 2024. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- FIKT21. Hiroki Furue, Yasuhiko Ikematsu, Yutaro Kiyomura, and Tsuyoshi Takagi. A new variant of unbalanced Oil and Vinegar using quotient ring: QR-UOV. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021*,

- Part IV*, volume 13093 of *LNCS*, pages 187–217. Springer, Cham, December 2021.
- FNT21. Hiroki Furue, Shuhei Nakamura, and Tsuyoshi Takagi. Improving Thomae-Wolf algorithm for solving underdetermined multivariate quadratic polynomial problem. In Jung Hee Cheon and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 12th International Workshop, PQCrypto 2021*, pages 65–78. Springer, Cham, 2021.
- GJ79. M. Garey and D. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco, 1979.
- Gro98. Lov K. Grover. A framework for fast quantum mechanical algorithms. In *30th ACM STOC*, pages 53–62. ACM Press, May 1998.
- Has21. Yasufumi Hashimoto. An improvement of algorithms to solve under-defined systems of multivariate quadratic equations. Cryptology ePrint Archive, Paper 2021/1045, 2021.
- KPG99. Aviad Kipnis, Jacques Patarin, and Louis Goubin. Unbalanced Oil and Vinegar signature schemes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 206–222. Springer, Berlin, Heidelberg, May 1999.
- MHT13. Hiroyuki Miura, Yasufumi Hashimoto, and Tsuyoshi Takagi. Extended algorithm for solving underdefined multivariate quadratic equations. In Philippe Gaborit, editor, *Post-Quantum Cryptography - 5th International Workshop, PQCrypto 2013*, pages 118–135. Springer, Berlin, Heidelberg, June 2013.
- MS17. Rusydi H Makarim and Marc Stevens. M4gb: an efficient gröbner-basis algorithm. In *Proceedings of the 2017 ACM International Symposium on Symbolic and Algebraic Computation*, pages 293–300, 2017.
- Pat97. Jacques Patarin. The oil and vinegar signature scheme. In *Presented at the Dagstuhl Workshop on Cryptography September 1997*, 1997.
- TW12. Enrico Thomae and Christopher Wolf. Solving underdetermined systems of multivariate quadratic equations revisited. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 156–171. Springer, Berlin, Heidelberg, May 2012.
- WCD⁺24. Lih-Chung Wang, Chun-Yen Chou, Jintai Ding, Yen-Liang Kuan, Ming-Siou Li, Bo-Shu Tseng, Po-En Tseng, and Chia-Chun Wang. SNOVA. Technical report, National Institute of Standards and Technology, 2024. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>.
- WTKC22. Lih-Chung Wang, Po-En Tseng, Yen-Liang Kuan, and Chun-Yen Chou. A simple noncommutative UOV scheme. Cryptology ePrint Archive, Report 2022/1742, 2022.